



Route planning with turn restrictions: A computational experiment

Stéphanie Vanhove*, Veerle Fack

Department of Applied Mathematics and Computer Science, Ghent University, Krijgslaan 281 - S9, B-9000 Ghent, Belgium

ARTICLE INFO

Article history:

Received 12 July 2011

Accepted 27 May 2012

Available online 6 June 2012

Keywords:

Routing
Turn cost
Turn prohibition
Graph algorithm
Shortest path

ABSTRACT

Real-world road networks often contain turn penalties or forbidden turns. Standard shortest path algorithms do not take these into account. Several ways of dealing with such turn restrictions have been proposed. It has remained unclear which method is most suitable, even though efficiency is very important in route planning. We present a computational experiment comparing these methods. We conclude with a guideline for choosing the right algorithm in a real-world application.

© 2012 Elsevier B.V. All rights reserved.

1. Introduction

Shortest path algorithms are very often used in route planners and mobile navigation systems. For this purpose, road networks are modelled as a graph, where nodes represent intersections or dead ends, arcs represent road segments, and weights typically represent either travel times or distances. However, this model is not always sufficient for a realistic road network. Usually not all turns can be taken in a road network, e.g., because of a lane divider or a forbidden turn sign, and even if a turn is legal, it usually implies an additional cost, e.g., the time needed to physically take a sharp turn, the time spent waiting at an intersection for other vehicles to pass, or the time spent waiting at stoplights. Fig. 1 shows two situations where a detour is the best option to avoid a forbidden or expensive turn. These examples clearly show that it is crucial for a route planner to take such turn restrictions into account. However, standard shortest path algorithms assume that every turn is legal and has a zero cost.

Different methods have been proposed to overcome this problem. They will be discussed more thoroughly in the following sections. The calculations must also be very efficient since route planners receive very large numbers of queries. It would be of interest to learn how these techniques compete with each other.

The number of turn prohibitions in a road network is of course dependent on the topology of the network. Furthermore, in a real-life situation not all datasets may store a cost for every turn in the network. This can be for memory reasons, but also since

visiting every turn to determine this cost would be a tremendous effort. Therefore, a dataset may store only the turns for the most important or busiest intersections. However, it is also possible that many or all turns are stored, e.g., if the turn costs are calculated automatically based on the angle. This motivates an experimental study to discover the influence of the amount of stored information on the performance of the algorithms. The results can be used as a guideline for choosing the best method for a particular dataset. While some studies comparing shortest path algorithms *without* turn restrictions can be found in the literature (e.g., [5,21,22,12]), no such study has been done for shortest path algorithms *with* turn restrictions until now.

We will consider both turn costs and turn prohibitions, using *turn restrictions* as the general term for both. A *turn* is any transition from one arc to another. This can be a left or right turn, a U-turn, or even straight through. Fig. 2 shows an example of a graph with turn prohibitions. The solid arrows represent arcs, which are always directed in order to represent one-way streets as well. The dashed arrows indicate turn restrictions, in this case forbidden turns. It is required that arc weights are always non-negative, which is a realistic assumption in road networks.

2. Literature overview

The literature considers two possible approaches for handling turn restrictions in shortest path algorithms. One approach consists in modifying the graph to integrate the information about the turn restrictions in the structure of the graph itself. Standard graph algorithms can then be applied to the modified graph, without even knowing about the turn restrictions. Examples of this approach are the node splitting method [11,18] and the line graph method [4,1,20,9].

* Corresponding author. Tel.: +32 9 264 48 94; fax: +32 9 264 49 95.

E-mail addresses: Stephanie.Vanhove@UGent.be (S. Vanhove), Veerle.Fack@UGent.be (V. Fack).

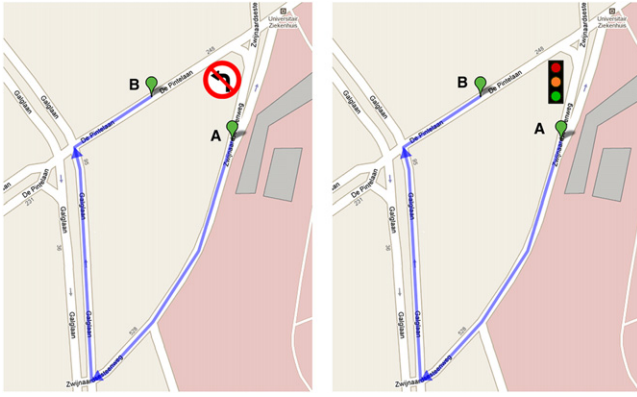


Fig. 1. Two situations where a standard shortest path algorithm would take the obvious left turn to go from *A* to *B*. In reality, however, the best route is the detour shown in the pictures because of either a forbidden turn (left) or a long waiting time at stoplights (right).

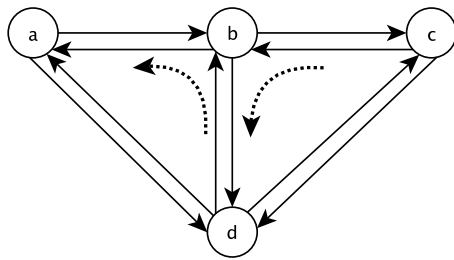


Fig. 2. Graph with forbidden turns. Dashed arrows indicate forbidden turns: it is legal to move from arc (d, b) to (b, c) but it is illegal to move from arc (d, b) to (b, a) .

Another approach is to develop a new algorithm which works on the original graph and explicitly takes the turn restrictions into account. We will call this approach the direct method. Gutiérrez and Medaglia [10] present a modified version of the well-known algorithm of Dijkstra [8], and Ziliaskopoulos and Mahmassani [23] present a similar method.

Besides the shortest path algorithms with turn restrictions themselves, it is also interesting to learn how turn restrictions are assigned to a network. Nielsen et al. [16] give a very detailed description of the calculation of turn costs, taking an impressive range of factors into account such as road capacity, road congestion, priority of the roads, green time at stoplights, and green waves. Another method for assigning turn costs is described by Volker [19].

Not only shortest path algorithms can be affected by turn restrictions, many graph algorithms can be applied to road networks and should take turn restrictions into account. Micó and Soler [13] present a very general heuristic and optimal method which can be used to include turn restrictions in many well-known graph algorithms such as the Chinese Postman Problem, the Travelling Salesman problem, and the Capacitated Arc Routing Problem. A more specific method is described by Bräysy et al. [3]. Clossey et al. [6] describe both a direct method and a graph transforming method for the Chinese Postman Problem.

Finally, an efficient implementation for shortest path calculation with turn prohibitions in the specific case where a turn is only legal if its angle is between certain values, is proposed by Boroujerdi [2].

3. The three studied algorithms

We have implemented and performed experiments with the node splitting method, the line graph method and the direct method, since all methods in the literature are instances of one of

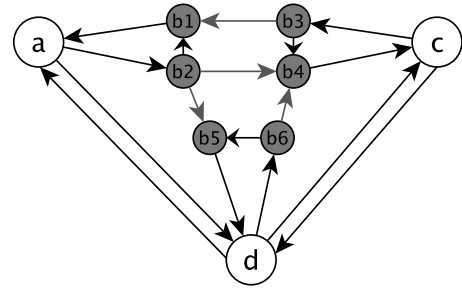


Fig. 3. Node splitting: transformed graph for Fig. 2. Node b is split into 6 nodes: $b_1, b_2, b_3, b_4, b_5, b_6$, each representing an arc incident with node b . Arcs between the split nodes represent legal turns. Split nodes and the arcs between them are shown in grey.

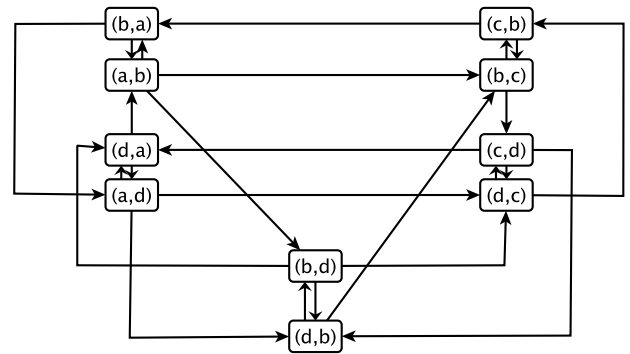


Fig. 4. Line graph: transformed graph for Fig. 2. Every node in the line graph represents an arc in the original graph. Arcs in the line graph represent legal turns in the original graph.

these three techniques. The following paragraphs describe these methods in more detail.

3.1. Graph transforming method: node splitting

Node splitting is an intuitive way to modify the graph. Fig. 3 shows an example of node splitting for the graph in Fig. 2.

Nodes with turn restrictions are split into several nodes, one node for every incoming or outgoing arc. Legal turns are represented by arcs between these nodes. The cost of such an arc is equal to the cost of the represented turn. Forbidden turns are represented by simply *not* adding an arc between the corresponding pair of nodes, making it impossible to take this turn. Nodes without turn restrictions are not split in the transformed graph. Therefore, the size of the transformed graph is dependent on the number of nodes with turn restrictions.

3.2. Graph transforming method: line graph

Another graph transforming method transforms the graph into its line graph. Fig. 4 shows the line graph for the graph in Fig. 2.

Every node in the line graph represents an arc in the original graph, while every arc represents a legal turn in the original graph. The weight of such an arc in the line graph is the sum of the turn cost and the weight of the arc which the turn leads to. As with node splitting, illegal turns are simply represented by *not* adding an arc between the two corresponding nodes. However, there is an important difference between the line graph and node splitting. As mentioned before, the node splitting method only splits nodes with turn restrictions, while the line graph method always transforms the entire graph, even if there are nodes without turn restrictions. Therefore, the number of nodes in the line graph is always equal to the number of arcs in the original graph, and the number of arcs in the line graph is always equal to the number of legal turns in the original graph.

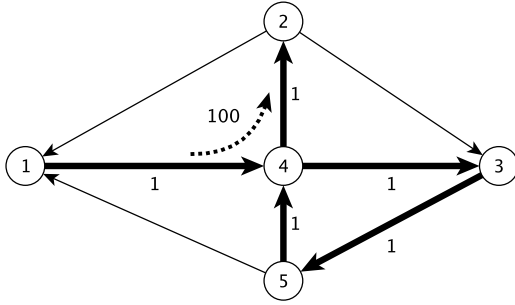


Fig. 5. Situation where the shortest route contains a cycle. The shortest route without cycles from 1 to 2 is 1–4–2 and has weight 102. However, the shortest route is 1–4–3–5–4–2 and has weight 5. This route has a cycle since node 4 is used twice. This example shows that, when turn restrictions are present, a shortest route can contain the same node more than once.

3.3. Algorithm for original graph: direct method

It is also possible to modify the algorithm instead of the graph. For example, Gutiérrez and Medaglia [10] present a modified version of the algorithm of Dijkstra for finding shortest paths with turn prohibitions. We will call this method the direct method. The algorithm of Dijkstra calculates a shortest path which never contains the same node twice, so there are no cycles in the resulting path. However, if the graph has turn restrictions, the shortest route can sometimes include cycles. An example of this situation is shown in Fig. 5.

While the algorithm of Dijkstra assigns labels to nodes representing the distance from the start node, the direct method assigns labels to arcs instead of nodes. In this way, the direct method calculates a shortest path which never contains the same arc twice, but which can contain the same node more than once. An arc label represents the distance from the start node to this arc, including the arc's weight itself.

While the authors only describe the direct method for use with turn prohibitions, we extend this method for accommodating turn costs as well, as described in the next paragraph.

The direct method starts by assigning a label to every outgoing arc of the start node, equal to the arc's weight, and a label with value infinity to all other arcs. However, every newly assigned label is only a *temporary* label since a shorter distance to this arc may still be found. In every iteration the algorithm selects the arc A with the smallest temporary label in the entire graph and makes this label *permanent*. The algorithm then visits all subsequent arcs A' of A . If A' already has a permanent label, A' is skipped. Otherwise, the algorithm looks up the turn restriction from A to A' . If this turn is forbidden, the arc A' is skipped as well. If the turn is legal, a potential new temporary label for A' is calculated which is the sum of the label of A , the turn cost from A to A' (which can of course be zero) and the weight of A' . If this new label is smaller than the current temporary label of A' , the label of A' is updated and a reference from A' to A is stored to facilitate the retrieval of the shortest path. It can be proven that when a label is permanent, no shorter path to this arc can be found, so the algorithm stops as soon as an arc incident with the target node receives a permanent label.

Since outgoing arcs and turn restrictions are looked up very often by the algorithm, an efficient data structure representing the graph is important. We use a data structure based on an adjacency list, i.e., a list which stores its neighbours for every node. It is similar to the data structure used by Gutiérrez and Medaglia, although the implementation differs slightly. Nodes are not represented by objects but by numbers from 0 to $n - 1$ (with n the number of nodes). This numbering allows easy indexing in lists. Arcs are stored as Java objects with the tail node, head node and weight of the arc as fields. Unlike Gutiérrez and Medaglia, our data structure

stores not only turn prohibitions, but also turn costs. For every arc, a Java HashMap is stored with its consecutive arcs and their corresponding turn cost. Forbidden turns are simply stored as an infinite turn cost. If a turn cost is zero, no entry is added to the map. This may save a lot of memory if a lot of legal turns have a zero cost. The use of a Java HashMap allows us to check whether a turn is legal and to retrieve a turn cost in constant time. This graph implementation is used by the direct method and serves as a basis for a graph transformation by the other methods.

It should be noted that this direct method is similar to a standard Dijkstra algorithm performed on the line graph. When the algorithm of Dijkstra labels a node in the line graph, it assigns a label to a node which corresponds to an arc in the original graph. This is equivalent to the labelling of an arc in the original graph by the direct method. We may therefore expect a more or less similar behaviour for both algorithms.

3.4. Time complexity

As mentioned by Cormen et al. [7], the algorithm of Dijkstra can be implemented with a time complexity of $O(m \log n)$ if all nodes can be reached from the start node, m being the number of arcs and n being the number of nodes. Let t be the number of turns in the graph, t_l the number of legal turns, t_f the number of forbidden turns, and t_c the number of turns with a non-zero turn cost. Let $t_r = t_f + t_c$ be the total number of turns with a turn restriction. Since the line graph method performs the algorithm of Dijkstra on a graph with m nodes and t_l arcs, the line graph method has a time complexity of $O(t_l \log m)$.

Even though the behaviour of the direct method is very similar to the line graph method, the complexity is slightly different. Our implementation of the direct method iterates over *all* turns and then checks whether they are legal or not. Illegal turns are then skipped while for legal turns, an $O(\log m)$ amount of work is done in this implementation. This results in a time complexity of $O(t + t_l \log m)$, which is in practice not substantially different from $O(t_l \log m)$.

The node splitting method splits every node which is restricted by a turn cost or turn prohibition into several nodes, one for every incoming or outgoing arc. The number of such restricted nodes in the original graph is bounded by $O(t_r)$, so the number of nodes after node splitting is bounded by $O(n + t_r d_{\max})$, where $d_{\max}$ is the maximum number of incoming or outgoing arcs in a node. An arc is added for every legal turn at a node with turn restrictions, so the number of arcs is bounded by $O(m + t_l)$. This leads to a time complexity of $O((m + t_l) \log(n + t_r d_{\max}))$.

3.5. Memory complexity

Our graph implementation stores one object for every arc. For the nodes, no specific object is stored except for a list containing all of its outgoing arcs. There are m objects representing arcs and n (empty or non-empty) lists of arcs. This leads to a memory complexity of $O(n + m)$ for a graph with no turn restrictions. When adding turn restrictions, an entry is stored in a HashMap for every restricted turn. This results in a memory complexity of $O(n + m + t_r)$ for a graph with turn restrictions as used by our implementation of the direct method. The line graph however is actually a transformed graph without any turn restrictions, so only its number of nodes m and its number of arcs t_l need to be taken into account. This leads to a memory complexity of $O(m + t_l)$ for the line graph. Finally, for node splitting, the new number of arcs is bounded by $O(m + t_l)$, so the memory complexity is $O(n + m + t_l)$. From this theoretical complexity analysis, it can be expected that t_r and t_l have an important influence on the performance of the algorithms, as the results will confirm.

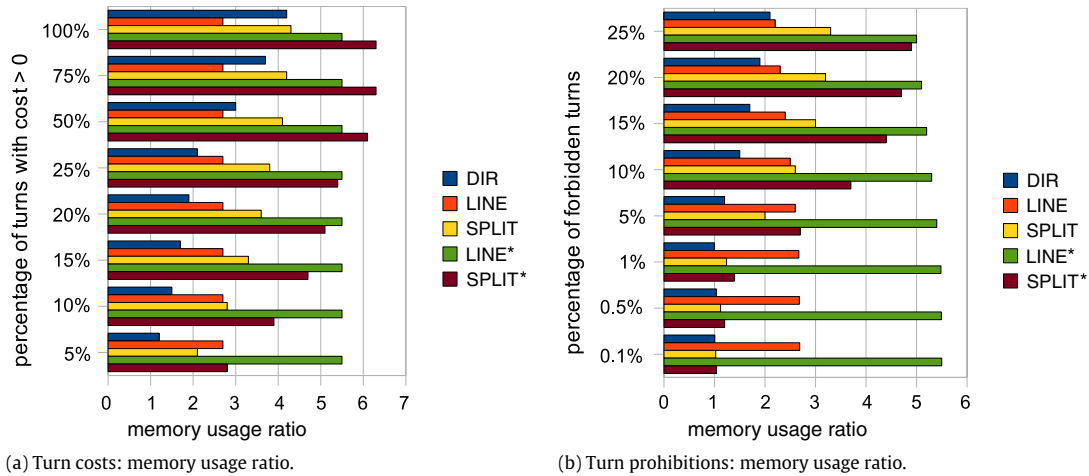


Fig. 6. Belgian road network: memory usage ratio for different percentages of turn costs (a) and turn prohibitions (b) for the direct method (DIR), line graph without lookup table (LINE), node splitting without lookup table (SPLIT), line graph with lookup table (LINE*) and node splitting with lookup table (SPLIT*). The graph has 458,403 nodes, 1085,076 arcs and 2922,504 turns. The size of the original graph without turn restrictions is 57.72 MB.

4. Lookup tables for transformed graphs

When dealing with transformed graphs, certain implementation issues arise. Any node in the original graph may have more than one corresponding node in the transformed graph. Hence, a shortest path query can have multiple start and/or target nodes in the transformed graph. However, a standard shortest path algorithm such as the algorithm of Dijkstra cannot deal with this situation. As proposed by Speičys et al. [18], this can be solved quite easily by adding a virtual start and target node to the transformed graph with arcs connecting them to all the possible start and target nodes, respectively. Also, after running the shortest path algorithm, the found path must be converted back to a path in the original graph. Taking this into account, a shortest path algorithm on a transformed graph consists of three steps:

1. Create virtual nodes in the transformed graph.
2. Run any shortest path algorithm on the transformed graph (e.g., the algorithm of Dijkstra).
3. Convert shortest path.

We will refer to the time spent for Step 2 as the *strict query time*. However, Steps 1 and 3 may take some time as well, since they both require finding nodes in the transformed graph corresponding to a node in the original graph or vice versa. This raises the question if it would be interesting to store a lookup table in memory in which these corresponding nodes can be looked up in constant time. While this decreases the complexity of Steps 1 and 3, it also requires extra memory. We aim to investigate the benefits and drawbacks of this approach. Therefore, we have implemented a version with and without a lookup table for both the line graph method and node splitting. For the direct method, such a lookup table is not necessary since no virtual nodes or path conversion are needed.

5. Computational experiments

In the experiments we aim to discover the influence of the amount of turn restrictions on the performance on the algorithms. We also investigate how the algorithms perform on a set of road networks with real-life turn prohibitions provided by Navteq. All algorithms were implemented, compiled and executed in Java version 1.6.0_03. All tests were run on an Intel dual core 2.13 GHz machine with 2 GB RAM. As mentioned before, we need to select a standard shortest path algorithm for calculating shortest paths in the transformed graphs. We chose the algorithm of Dijkstra, since

virtually every route planning application is based on some variant of the algorithm of Dijkstra, but we expect similar behaviour for other shortest path algorithms.

5.1. Influence of the amount of turn restrictions

For a fair comparison, only the amount of turn restrictions should be variable while all other factors should remain constant. For this purpose we applied different amounts of random turn restrictions to each of our test graphs. While the turn restrictions are random, the test graphs do represent real-life road networks for European countries. The data are provided by the Institute of Theoretical Informatics at the Karlsruhe Institute of Technology (KIT), led by Dorothea Wagner. The data were provided to the Institute of Theoretical Informatics by the PTV Company in Karlsruhe [17]. In this paper we present detailed results for the Belgian road network. This graph has 458,403 nodes, 1085,076 arcs, and 2922,504 turns. Arc weights represent Euclidean travel distances. Similar results were obtained for the road networks of Switzerland, Ireland, Luxembourg, and Portugal. We do not report the details in this paper.

We did experiments where 5%, 10%, 15%, 20%, 25%, 50%, 75% and 100% of the turns in the same graph have a randomly chosen non-zero turn cost. For left turns we chose random costs between zero and three times the average arc weight taken over all arcs in the graph. Since waiting times for right turns are usually smaller, right turns were assigned random turn costs between zero and 1.5 times the average arc weight. Since forbidden turns are less common in road networks, lower percentages are chosen for forbidden turns. In this case, different versions of the same graph were assigned 0.1%, 0.5%, 1%, 5%, 10%, 15%, 20% and 25% randomly chosen forbidden turns.

5.1.1. Memory usage

For each of our European networks, we have measured the size in memory of the following: the original graph without any turn restrictions, the original graph + turn restrictions data for the direct method (DIR), the line graph without lookup table (LINE), the line graph with lookup table (LINE*), the transformed graph used by the node splitting method without lookup table (SPLIT) and the transformed graph used by the node splitting method with lookup table (SPLIT*).

The results for the Belgian road network can be seen in Fig. 6. The displayed values are not absolute values, but ratios compared to the original graph without any turn restrictions. For example, for

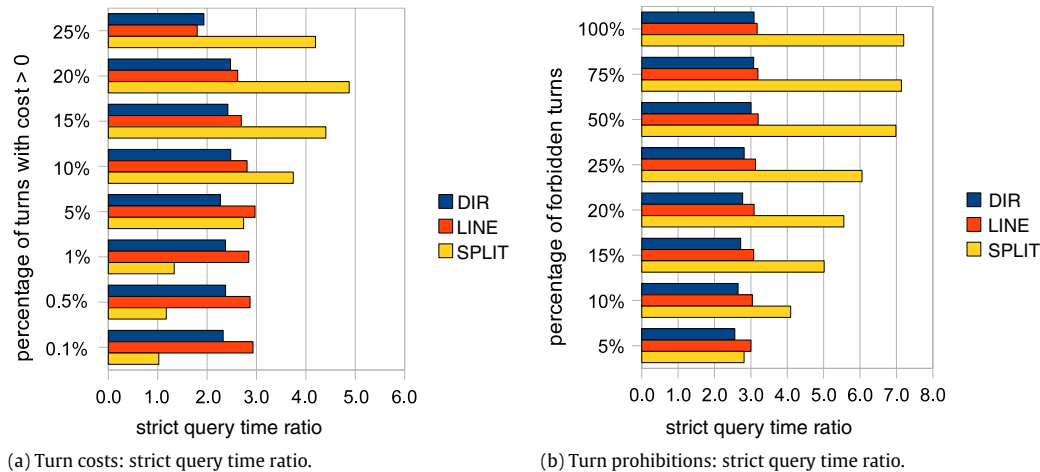


Fig. 7. Belgian road network: average strict query time ratio for different percentages of turn costs (a) and turn prohibitions (b) for the direct method (DIR), the line graph (LINE) and node splitting (SPLIT). The graph has 458,403 nodes, 1085,076 arcs and 2922,504 turns. The average running time for a one-to-one query of the standard Dijkstra algorithm on the original graph is 254.49 ms.

a graph where 20% of the turns imply a non-zero cost, the direct method takes 1.9 times more memory than the original graph. From these charts it is clear that using a lookup table requires a significant amount of extra memory, sometimes even up to a factor of two.

In the case of turn costs, memory usage for the line graph is constant, since this method always transforms the entire graph, even if there are only a few turn costs. On the other hand, memory usage for the direct and node splitting methods increases with the number of non-zero turn costs, since more information on turn costs needs to be stored and more nodes need to be split, respectively. Up to about 25% turn costs, the direct method requires the smallest amount of memory, while for graphs with more turn costs, the line graph (without lookup table) is more memory-efficient. In the case of turn prohibitions, the direct method is always most memory-efficient for the tested percentages. However, for very small percentages of turn prohibitions, the memory-efficiency of the node splitting method and direct method are similar, with the direct method clearly taking over around 5%. The direct method shows exactly the same behaviour for turn costs and turn prohibitions, since saving information on forbidden turns and turn costs takes the same amount of memory. However, memory usage for the line graph decreases with the number of turn prohibitions. This can be explained by the fact that fewer legal turns means fewer arcs in the line graph. As for node splitting, memory usage increases with the number of turn prohibitions, since more nodes need to be split. However, the increase is slower for higher numbers of turn prohibitions since more nodes are already split.

5.1.2. Running times

For the time measurements a set of one hundred random one-to-one queries was generated for each of our European road networks. Every query in such a set has a random start node and a random target node. For each graph, time measurements were performed with different parameters. These measurements were all performed on the same set of one hundred queries which was generated particularly for this graph. The parameters include: the method which was used, the type of turn restrictions (costs or prohibitions) and the number of turn restrictions. The average running time was calculated over these one hundred queries for each set of parameters. Furthermore, even the same query with the same set of parameters was executed 10 times and its average was taken, just to make sure that the time measurements are accurate. Loading the graph into memory is never included in the

time measurements since this needs to happen only once for all the experiments on a graph. Fig. 7 displays the results. They only concern the *strict query time*, i.e., Step 2 mentioned in Section 4. The running times for the other two steps were measured separately and will be discussed later. Also, for the time measurements no implementations using a lookup table are considered, since such a lookup table is not used in this step of the algorithm.

The displayed values in Fig. 7 are again ratios compared to the average running time for the same queries for a standard Dijkstra algorithm on the original graph. Both charts show very similar query times for the direct method and the line graph, which can be explained by the similarities between both methods. For turn costs, the direct method and the line graph method clearly outperform node splitting. For turn prohibitions, node splitting is the fastest method for few turn prohibitions (less than 5%), but for road networks with many turn prohibitions, the direct method and line graph method are again the fastest. Since their running times are so similar, the actual choice between the direct method and the line graph method should be made based on memory usage.

5.1.3. Time needed for creating virtual nodes and path conversion

Running times for creating virtual nodes (Step 1) and path conversion (Step 3) were measured separately. For almost all methods, regardless of the usage of a lookup table, the average running times for Step 1 and Step 3 were below 1 ms. Keeping in mind that a one-to-one query of the standard Dijkstra algorithm on average takes 254.49 ms for this graph and that the running times measured for the evaluated methods are a multiple of this, a running time smaller than 1 ms is definitely insignificant. However, there is one exception. The average running time for creating virtual nodes in the line graph method (without a lookup table) is around 17 ms. While the lookup table does provide some speedup here, it is rather small compared to the large amount of extra required memory and is generally not worth it unless gaining even a small amount of time is very critical.

5.2. Performance on real-world road networks with turn prohibitions by Navteq

The algorithms were also tested on road networks with real-world turn restrictions. We had to limit ourselves to turn prohibitions only since, to the best of our knowledge, no datasets with real-life turn costs are available at this moment. The road networks with real-world turn prohibitions are provided by

Table 1

Navteq real-world road network characteristics. The table shows the size and amount of turn prohibitions (% TP) for every road network. The column 'Dijkstra' represents the average time in milliseconds for a shortest path calculation using the algorithm of Dijkstra.

Road network	# Nodes	# Arcs	# Turns	TP (%)	Size (MB)	Dijkstra (ms)
Luxembourg	39,883	89,594	226,840	0.14	4.85	12.03
Belgium	564,477	1,300,765	3,403,729	0.10	69.18	389.62
The Netherlands	1,017,242	2,407,244	6,496,909	0.04	128.16	783.35
Paris	313,536	705,588	1,809,714	0.57	38.04	151.00

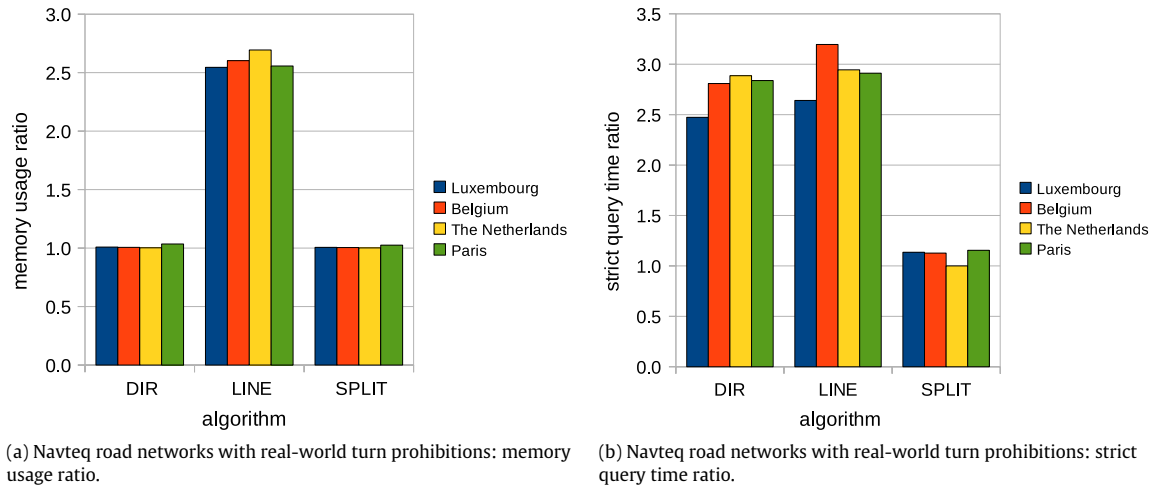


Fig. 8. Results for the Navteq road networks with real-world turn prohibitions. Memory usage ratio (a) compared to the original graph without turn restrictions and strict query time ratio (b) compared to the average time needed for running the algorithm of Dijkstra on the original graph. The absolute numbers for the original graphs can be found in Table 1.

Navteq [14,15], a major American provider of GIS data and digital street maps.

The results confirm the results presented in the previous paragraphs. Fig. 8 shows the memory usage ratio and strict query time ratio for the four Navteq networks. More information on these road networks can be found in Table 1. As can be seen in the table, these road networks have very low percentages of turn prohibitions, something which can be expected in a realistic road network. Only results for implementations without a lookup table are shown.

The results clearly confirm that node splitting is the fastest method, and is also very memory-efficient. Furthermore, given the low percentages of turn prohibitions, the ratios for both memory and running times are very similar to the ratios obtained in the experiments in Section 5.1. It should also be noted that the results show absolutely no significant difference between Paris, an urban road network, and the road networks representing entire countries.

5.3. Guideline for real-world applications

The results can be summarised in the following guideline for real-world applications. For road networks with turn costs, use the direct method if no more than 25% of the turns have a non-zero cost, and use the line graph method otherwise. For road networks with forbidden turns, use the node splitting method if less than 5% of the turns are forbidden, which is generally the case. In exceptional cases where 5% of the turns or more are forbidden, use the direct method. Using a lookup table with additional information is generally not worth the extra memory since the speedup is rather insignificant.

Acknowledgment

SV is supported by a research grant of the Research Foundation Flanders.

References

- [1] J. Anez, T. De La Barra, B. Perez, Dual graph representation of transport networks, *Transportation Research Part B: Methodological* 30 (1996) 209–216.
- [2] A. Boroujerdi, An efficient algorithm for computing least cost paths with turn constraints, *Information Processing Letters* 67 (1998) 317–321.
- [3] O. Bräysy, E. Martínez, Y. Nagata, D. Soler, The mixed capacitated general routing problem with turn penalties, *Expert Systems with Applications* (2011).
- [4] T. Caldwell, On finding minimum routes in a network with turn penalties, *Communications of the ACM* 4 (1961) 107–108.
- [5] B. Cherkassky, A. Goldberg, Shortest paths algorithms: theory and experimental evaluation, *Mathematical Programming* 73 (1996) 129–174.
- [6] J. Clossey, G. Laporte, P. Soriano, Solving arc routing problems with turn penalties, *Journal of the Operational Research Society* 52 (2001) 433–439.
- [7] T. Cormen, C. Leiserson, R. Rivest, *Introduction to Algorithms*, MIT Press, 2005.
- [8] E.W. Dijkstra, A note on two problems in connexion with graphs, *Numerische Mathematik* 1 (1959) 269–271.
- [9] M. Fidler, G. Einhoff, Routing in turn-prohibition based feed-forward networks, in: *NETWORKING 2004*, Networking Technologies, Services, and Protocols; Performance of Computer and Communication Networks; Mobile and Wireless Communications, 2004, pp. 1168–1179.
- [10] E. Gutiérrez, A.L. Medaglia, Labeling algorithm for the shortest path problem with turn prohibitions with application to large-scale road networks, *Annals of Operations Research* 157 (2008) 169–182.
- [11] R. Kirkby, R. Potts, The minimum route problem for networks with turn penalties and prohibitions, *Transportation Research* 3 (1969) 397–408.
- [12] U. Lauther, An experimental evaluation of point-to-point shortest path calculation on road networks, in: *The Shortest Path Problem: Ninth DIMACS*, 2006, pp. 1–18.
- [13] J.C. Micó, D. Soler, The capacitated general windy routing problem with turn penalties, *Operations Research Letters* (2011) 1–7.
- [14] Navteq, www.navteq.com, 2011.
- [15] Navteq network for developers, www.nn4d.com, 2011.
- [16] O. Nielsen, R. Frederiksen, N. Simonsen, Using expert system rules to establish data for intersections and turns in road networks, *International Transactions in Operational Research* 5 (1998) 569–581.
- [17] PTV, Ptv. company Karlsruhe, <http://www.ptv.de/>, 2005.
- [18] L. Speičys, C.S. Jensen, A. Kligys, Computational data modeling for network-constrained moving objects, in: *Proceedings of the Eleventh ACM International Symposium on Advances in Geographic Information Systems—GIS 2003*, 2003, pp. 118–125.
- [19] L. Volker, Route planning in road networks with turn costs, *Studienarbeit*, Universität Karlsruhe, Institut für Theoretische Informatik, 2008, pp. 1–30.

- [20] S. Winter, Modeling costs of turns in route planning, *GeoInformatica* 6 (2002) 345–361.
- [21] F. Zhan, Three fastest shortest path algorithms on real road networks: data structures and procedures, *Journal of Geographic Information and Decision Analysis* 1 (1997) 69–82.
- [22] F. Zhan, C.E. Noon, Shortest path algorithms: an evaluation using real road networks, *Transportation Science* 32 (1998) 65–73.
- [23] A. Ziliaskopoulos, H. Mahmassani, A note on least time path computation considering delays and prohibitions for intersection movements, *Transportation Research Part B: Methodological* 30 (1996) 359–367.